Assignment-5

Git Commit Requirements

Your Git history should show the progression of your work, not just the final solution.

What is required

- Make a commit after each meaningful change or milestone (creating a class header or source file, implementing a substantial class method, completing one part of the assignment, adding a new feature, or fixing a significant bug).
- Each commit message must briefly explain what you implemented, fixed, or changed.
- Commit regularly while working rather than committing the entire assignment at the end.
- Create commits as you work. Do not complete the assignment first and then divide the finished code into several commits afterward.
- You do not need to push after every commit. You may commit your changes locally and push all commits later.

Good commit examples

- `Implemented input validation in main function`
- `Fixed incorrect calculation in method getValue`
- `Updated Car class constructor`
- `Corrected output formatting for method print`
- `Implemented toString method in House.cpp`

Not meaningful

- `commit 1`
- `update`
- `test`
- `done`
- `final`
- Multiple commits that contain no meaningful change

Grading

Git-related deductions are not cumulative. The highest applicable deduction will be used.

- **Up to -20% — Poor commit quality**: You made multiple commits, but the messages are vague, repetitive, or do not clearly describe your changes, or your commits do not consistently represent meaningful progress.
- **Up to −40% — Insufficient development history:** Your Git history does not reasonably demonstrate incremental development. Examples include submitting most or all of the assignment in one or a few very large commits, making nearly all significant commits immediately before the deadline, creating several commits only after the code was already

completed, or committing a large amount of new code only a few minutes after the previous commit.

A large number of commits by itself does not satisfy this requirement. Your commits should reflect actual, meaningful progress in your code.

Problem-1 (6 points):

The program aims to illustrate the concepts of shallow copy and deep copy in C++ using a **Person** class. You are asked to split class implementation from class definition.

- Create a class named **Person** to represent an individual with attributes **name** and **age**.
- Please use private int age and private char pointer for the name. **You are not allowed to use string or char array.**
- The class should include constructors, a destructor, and a display method.
- Implement a constructor that initializes a **Person** object with a name and an age. Inside this constructor, allocate memory for the name and copy the provided name into it.
    - **Person(const char* name, int age);**
- Implement copy constructor for deep copy, which allocates new memory for the name and copies its content.
    - **Person(const Person& other);**
- Define a destructor that frees the memory allocated for the **name** attribute.
- Implement a constant method named **display** to display the name and the age of the **Person** object.
- Write a test program (mainPerson.cpp) that:
- Dynamically creates a Person object on the heap using the constructor with the following arguments (“Clementine”, 21) and assigns the object address to the pointer pPer.
- Creates another object on the heap using a copy constructor and assigns the object address to the pointer pPer2.
- Deletes the memory allocated for pPer object.
- Displays the name and age of pPer2.
- Deletes the memory allocated for pPer2 object.

Here is a sample run:



Problem-2 (7 points):

Design a class named MySet which can hold integers from 0 to a defined range. The set is represented as a dynamic array of Boolean values. Element with the index j is true if integer j is in the set. The element with the index k is false if integer k is not in the set. The class contains:

- Private unsigned int data members range that represents the biggest possible integer in a set.
- Private size that represents a number of elements in a set.
- Private pointer to bool named set.
- A user-defined constructor MySet(unsigned) that creates a MySet object with dynamic array with the range of elements from 0 to the given number inclusively and initializes the set of elements to false.
- A copy constructor MySet(const MySet &) that creates a set with a given MySet object.
- A destructor that destroys dynamically allocated array.
- A function void insertElement(int k) that places a new integer k into a set by setting a[k] to true. Function should check if the given number is in the range of a set numbers and print a warning message if the given number is out of the range.
- A function void deleteElement(int m) that deletes integer m by setting a[m] to false. Function should check if the given number is in the range of a set numbers and print a warning message if the given number is out of the range.
- A constant function void printSet() that prints a set as a list of coma-separated numbers. The function prints only those elements that are present in the set. It also prints **{ --- }** for an empty set. You should implement the required output format (see the sample run).
- A constant function bool isEqualTo(const MySet&) that determines whether two sets are equal.

Write a test program (mainMySet.cpp) that does the following:

1. Creates a first MySet object with the range of numbers [0; 20].
2. Inserts numbers 13, 7, 20, 21.
3. Calls printSet function from the created object.
4. Deletes numbers 22 and 13.
5. Calls printSet function from the created object.
6. Creates a second MySet object with the range of numbers [0; 100].
7. Calls printSet function from the second object.
8. Creates a third MySet object with the copy constructor with the first object as an argument.
9. Checks is the third set equals to the first set by calling isEqualTo function from one of the objects and prints the result.

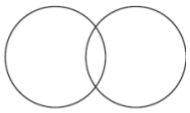
Here is a sample run:

```
Warning: cannot insert element 21 into MySet, it is out of range (not between 0 and 20 inclusive).
{ 7, 13, 20 }
Warning: cannot delete element 22 from MySet, it is out of range (not between 0 and 20 inclusive).
{ 7, 20 }
{ --- }
set3.isEqualTo(set) = 1
```

Problem-3 (7 points):

Design a class named Circle2D that contains:

1) Two double data fields named x and y that specify the center of the circle with constant get functions.
2) A double data field radius with a constant get function.
3) A static int data field numberOfCircles with a static get function getNumberOfCircles().
4) A constructor that creates a circle with the specified x, y, and radius. (Do not forget to update numberOfCircles).
5) A copy constructor Circle2D(const Circle2D & circle) that creates a circle with a given Circle2D object. (Do not forget to update numberOfCircles).
6) A destructor that updates numberOfCircles data field.
7) A constant function overlaps(const Circle2D& circle) that returns true if the specified circle overlaps with this circle. See Figure 1c.



(c)

Note: The distance from one point to another point is $\sqrt{(x-x2)^2+\ (y-y2)^2}$

Write a test program (mainCircle2D.cpp) that does the following:

1) Dynamically creates a Circle2D object on the heap using the constructor with the following arguments (2, 2, 5.5) and assigns the object address to the pointer pCir1.
2) Creates a Circle2D object cir2(-6, -3, 1.5).
3) Calls getNumberOfCircles() function from the pCir1.
4) Creates a Circle2D object cir3 with the copy constructor with the cir2 object as an argument.
5) Calls getNumberOfCircles() function from the cir3.
6) Calls overlaps(const Circle2D& circle) function from pCir1 pointer with the cir2 as an argument.
7) Calls overlaps(const Circle2D& circle) function from cir3 object with the cir2 as an argument.
8) Frees the memory allocated dynamically.
9) Calls getNumberOfCircles() function.

Here is the sample run:

```
Number of objects is 2
Number of objects is 3
pCir1 overlaps cir2 = 0
cir3 overlaps cir2 = 1
Number of objects is 2
```